

Towards RL Agents for Clash Royale: A Faithful and Efficient Game Simulator

1st Jason Jia

USTC

Hefei, China

jiahaoxiang@mail.ustc.edu.cn

Abstract—Clash Royale is an extremely popular real-time strategy mobile game, having a large observation space, action space and complex intrinsic card mechanics. A typical game in Clash Royale takes 3 minutes, meaning an agent must play thousands of full games to accumulate sufficient experience, making real-time training prohibitively slow, and the complexity of game and card mechanics has hindered simulator development, leaving no publicly available simulator that supports efficient RL training. We present a tick-based simulation framework that faithfully reproduces core gameplay mechanics - troop spawning, target locking and path finding - with 47 of 121 cards currently implemented. The simulation speed is 83 μ per tick and is 49x faster than real time, which fundamentally enables RL agents to learn to play the game within a reasonable timeframe, opening a door to self-play and strategy optimization research.

I. INTRODUCTION

Reinforcement learning has achieved remarkable success in game-playing fields such as mastering Atari games [1], defeating top human players in Dota 2 [2] and Starcraft II [3]. The success of RL in these games is largely attributed to the availability of efficient game simulators that allow the agents to accumulate millions of episodes of experience in a short period of time. Without such simulators, RL training would be bottlenecked by game speed, making complex games infeasible for training.

Clash Royale is a popular mobile strategy game with approximately 45.8 million monthly active players as of October 2024, according to third-party estimates [4]. In the game span of 3 minutes, each player accumulates elixir and deploys their cards in hand as troops into the arena. Each card has different stats and unique mechanics, with many different tiles to deploy on, resulting in a rich action space. The game requires quick decision making, placement evaluation and elixir management. However, despite its popularity, researchers in the RL community have largely overlooked this game partly due to the lack of a precise and efficient simulator. Training directly inside the game client or from previous game recordings has yielded poor performance because the agent cannot accumulate enough experience to make meaningful decisions.

In this paper, we present a lightweight, tick-based simulation framework for Clash Royale that reproduces core gameplay mechanics. Our simulator models troop spawning, spatial pathfinding with obstacle avoidance, target acquisition and locking, projectile creation, hitbox-based collision, and elixir management. We currently support 47 of the game's 121

cards, spanning a diverse range of unit types including melee troops, ranged attackers, flying units, and tanks. The simulator processes each game tick in 83 μ s on average, achieving a 48x speedup over real time and enabling hundreds of complete games to be simulated per hour on a single CPU core.

Our main contributions are as follows:

- 1) A faithful open-source Clash Royale simulator designed from the ground up for RL training, implementing core game mechanics with data extracted directly from the game client.
- 2) A high-performance tick-based architecture that achieves a 49x real-time speedup, removing environment speed as a bottleneck for agent training.
- 3) A Gym-compatible RL interface that supports standard training pipelines including self-play, enabling researchers to immediately begin agent development.

II. RELATED WORK

Simulator environments are fundamental to game-playing agent research, as they enable agents to accumulate experience orders of magnitude faster than real-time interaction. A seminal example is DQN [1], which achieved human-level performance on 49 Atari games by relying entirely on the Arcade Learning Environment (ALE) for high-speed experience collection. This reliance on simulators only deepened as games grew more complex. Neither OpenAI Five [2] on Dota 2 nor AlphaStar [3] on StarCraft II would have been feasible without high-speed simulation. OpenAI Five accumulated the equivalent of 45,000 years of gameplay during training.

Research on Clash Royale gameplay is bottlenecked by the lack of a simulator. First, the only known realistic battle engine is proprietary to Supercell, constraining any simulation to real-time speed and making it inaccessible to external researchers [5]. Second, while at least one independent research group developed a private simulator [6], it was never released, making their results impossible to reproduce. As a result, researchers without access to a simulator were forced to rely on computer vision-based approaches operating on real-time game footage [7], which severely limits experience collection. Wu et al. report a win rate of less than 5% against the in-game AI, far below human-level performance.

III. SYSTEM DESIGN

A. Tick-Based Architecture

Figure 1 gives a compact view of the simulator modules before the details of each component are introduced.

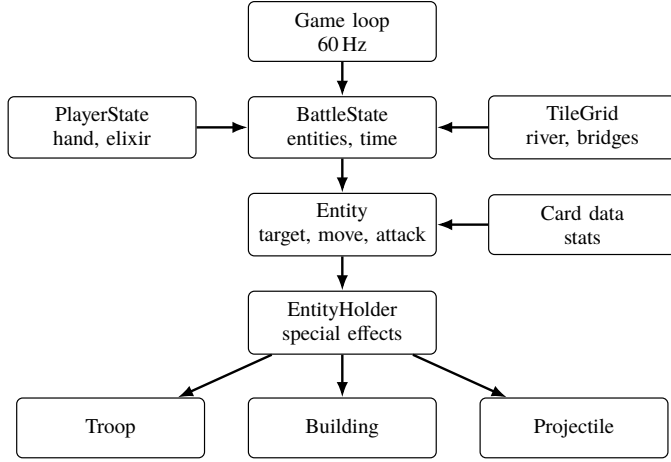


Fig. 1. Overall simulator architecture.

The simulator uses a tick based architecture that updates the game state 60 times per second in real life. Inside each tick, several core operations are performed. The game loop needs to handle gameplay logic including character spawning, cleaning dead entities, updating elixir and player's hands and evaluating win conditions - tower destroyed or timeout. Within each tick, the custom battle engine iteratively updates the state of every entity that's alive in the arena. This architecture affords the simulator greater modularity because the game logic and card logic can be separated and developed independently, with the game handling deployment, collisions and elixir management, while each card can follow its own rules of targeting, pathfinding and implementing their own spawn and death effects.

A significant advantage of using this architecture is that each tick executes with minimal overhead when not rendered on the screen. Under typical gameplay conditions, the time to run each tick on average takes 83 microseconds. After running the simulator on twenty games on a single laptop CPU core, where both players empty their elixir (ensuring maximum arena occupancy), the simulator took 337.8 seconds to run 100 games, where in real life the 100 games would actually span 16700.0 seconds, corresponding to approximately a 49.5x speedup over real time.

This significantly speeds up experience collection, rendering reinforcement learning training on Clash Royale practically feasible. To collect 1M frames of experience requires approximately 5 hours of real gameplay, but in a simulator, the actual time to collect the same amount of observation only needs approximately 5 minutes.

B. Pathfinding and Unit behaviour

The pathfinding rule is shown in Figure 2. The figure is only a schematic view of the rule-based movement logic.

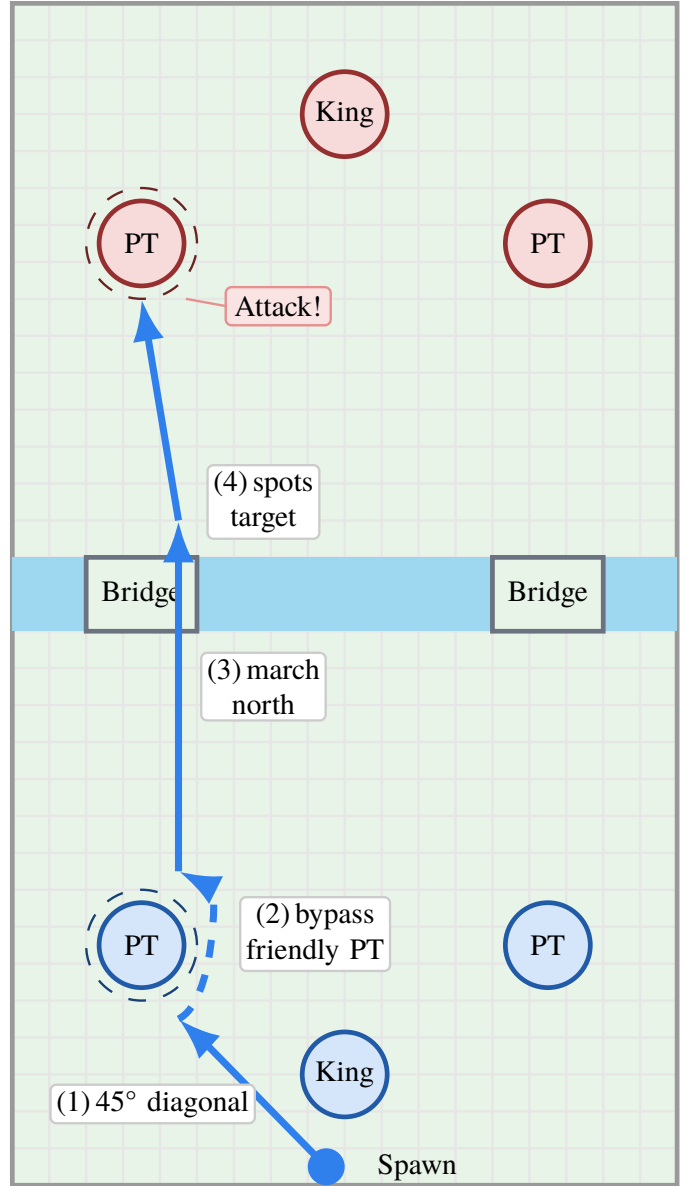


Fig. 2. Pathfinding example: a ground unit spawns at (8.5, 0.5) behind the friendly King Tower, moves diagonally to align with the left bridge lane (1), detours around the friendly Princess Tower collision radius (2), then advances straight north till it crosses the bridge (3), finally walks towards the enemy princess tower (4).

Pathfinding in Clash Royale is non-trivial, involving numerous subtle edge cases, and reverse-engineering the exact pathfinding algorithm from observation alone is infeasible.

However, the movement behaviour follows a set of consistent rules. When a character is deployed, if it is an air troop, then it flies in a straight line towards the nearest enemy tower, and if it is a ground troop, then it advances at a 45 degree angle to align with the nearest lane connected to a bridge and then walks forward. When the character spots a target, it walks in a straight line towards that target and attacks when in attack range. The unit's path becomes more complex when

it is blocked by an obstacle (e.g. a building), then the unit adjusts its heading incrementally to move around that obstacle and continue on its pre-set path.

The official game likely employs a more sophisticated pathfinding algorithm such as A* search algorithm for characters, which incurs substantial computational overhead. In each tick, every character’s route has to be re-calculated, and when there are many entities in the arena, the simulation speed will be significantly lowered. To avoid this overhead, the custom simulator adopts a rule-based approach. When the character moves according to simple rules without any planning, then in most cases the movement behaviours are still the same, and the time complexity is $O(1)$, reducing overhead and improving simulation throughput.

We note that unit movement around obstacles remains slower than observed in the original game client, which we identify as a limitation of the rule-based approach.

C. Special Effects System

Figure 3 illustrates the nested entity approach described below.

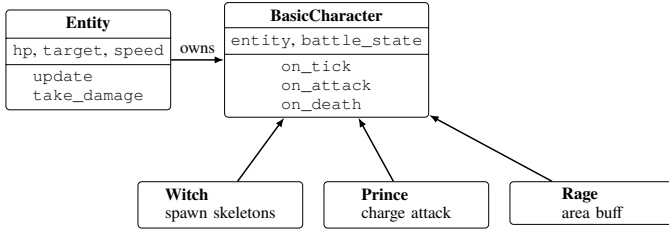


Fig. 3. EntityHolder structure for implementing special effects.

The most implementation-intensive aspect of the simulator is card behaviour. There are 121 cards in Clash Royale, each with different statistics and mechanics. Cards such as the Knight and Archers require minimal special logic, as their behaviour consists solely of movement and basic combat. However, numerous cards rely on entirely distinct mechanics. For example, when Lumberjack dies, it spawns a rage spell which deals a small amount of instant damage, increases the movement speed and attack speed of friendly troops in range. The witch spawns four skeletons around itself every 7 seconds and shoots projectiles that deal area damage. We refer to these as special effects.

A naive approach would be to subclass every card, but this leads to an unmanageable proliferation of subclasses, compromising maintainability. Here we propose a design method to dramatically simplify card implementation, which we call the nested entity approach.

In the simulator implementation, every entity inherits from the `Entity` class, containing methods for pathfinding, target locking and state update. Cards with special effects still largely adhere to these rules but their attacking logic is different. We added a new class to the simulator called `EntityHolder`, which stores the entity and battle state’s information and has its own `on_tick` function that is called every tick from the

original entity. With complete knowledge and control of the game state, special effects can be implemented directly within these functions.

For the witch’s periodic spawn effect to be implemented, it suffices to maintain a `next_spawn_time` attribute in `EntityHolder`, and the `on_tick` function can directly modify the game state to deploy skeletons near the witch’s position.

For the prince’s charging effect to be implemented, we need to rewrite both `on_tick` and `on_attack` functions to store the charging state and reset the charging state when the prince hits a target. This can be achieved concisely, allowing implementation to focus purely on card logic without having to modify the core battle engine, massively simplifying the codebase.

IV. RL ENVIRONMENT

After designing and implementing the Clash Royale simulator, we can deploy cards and observe their interactions within the simulator. However, a Gymnasium environment is required before RL training can begin, as it defines the observation space, action space, and reward function.

To support rapid training of AI agents, we designed a Gymnasium environment with custom observation space, action space and reward function. Figure 4 shows the components of the observation.

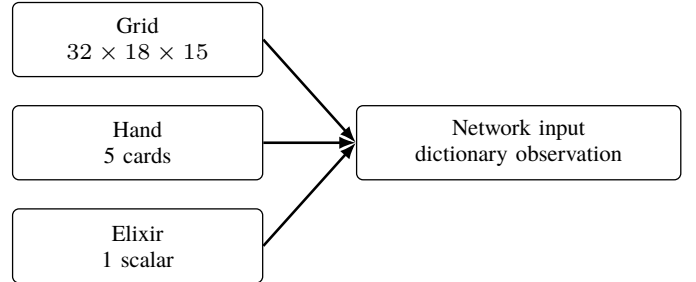


Fig. 4. Observation space used by the Gymnasium environment.

- The observation space has three components. `Grid` is a tensor with shape $(32, 18, 15)$, with the first two dimensions representing the arena, and the last dimension contains information describing the troop/building information on that tile. This is valid since collision radii prevent two entities from occupying the same tile simultaneously. The second component `Hand` encodes the agent’s currently available cards, and lastly `elixir` gives information on the current elixir of the player to support elixir management.
- The action space is relatively simple, consisting of only three parameters: `slot`, `y` and `x` referring to which card the player wants to deploy and the exact position on the arena. `slot` is an integer ranging from 0-4, with 0 meaning to stay idle.
- The reward function uses the following formula. The agent makes a decision twice every second and the reward

is computed as the difference between two consecutive game states. The agent receives a large reward or penalty upon destroying or losing a tower, and a small reward proportional to tower health advantage.

$$\begin{aligned}
R = & 0.001(\Delta HP_{\text{enemy}} - \Delta HP_{\text{self}}) \\
& + 5(\Delta Tower_{\text{enemy}} - \Delta Tower_{\text{self}}) \\
& + \begin{cases} +10, & \text{Win} \\ -10, & \text{Loss} \end{cases}
\end{aligned} \quad (1)$$

Because gymnasium environments are naturally compatible with stable-baselines3, a popular python library that supports out-of-the-box RL training, we trained a baseline PPO model to validate the environment.

The model structure is shown in Figure 5.

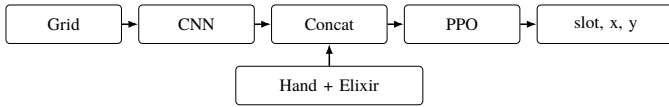


Fig. 5. Baseline PPO model structure.

In the model, we further processed the observation, replacing `card_id` with embeddings of length 8 and applying one-hot encoding to other categorical features, leading to the expansion of channels. We then used a simple convolutional neural network to process the spatial information, before concatenating the output with elixir and hand information as the final vector input to the PPO algorithm.

It is worth noting that the PPO algorithm produces the slot, y and x of the action space independently. We specifically designed the model this way to save computation and lower model complexity. If the agent generates all three values in the same network, then there are $5 \times 32 \times 18 = 2880$ actions total, making exploration significantly harder. The current model architecture is preliminary; a more principled design is left for future work.

V. EXPERIMENTS

A. Simulator Benchmark

This experiment stress-tests the simulator’s speed. The two players choose a random action every 0.5s, ensuring maximum arena occupancy. Then the evaluation program measures the time elapsed in real life and the equivalent time in game. In an evaluation over 100 simulated games using this strategy, it took 337.8 seconds in real life, where the equivalent in-game time is 16700s, corresponding to approximately a 49.5x speedup over real time. This implies that experience equivalent to 5 hours of real gameplay can be collected in approximately 6 minutes using the simulator, and the wall-clock time required to collect 1M timesteps of experience decreases from nearly 6 days to only 2.8 hours.

All of this was run on a MacBook laptop using a single CPU core. The simulation can be further sped up by using more cores and spawning multiple processes to collect experience.

TABLE I
SIMULATOR THROUGHPUT BENCHMARK.

Metric	Value
Games simulated	100
Equivalent game time	16700 s
Wall-clock time	337.8 s
Average tick cost	83 μ s
Real-time speedup	49.5x
Implemented cards	47 / 121

B. Model Benchmark

We evaluated the trained agent (1.6M timesteps) across 100 matches against a random-strategy opponent. The trained agent won 83 of 100 matches. This indicates that the agent has acquired non-trivial game knowledge enabling consistent wins.

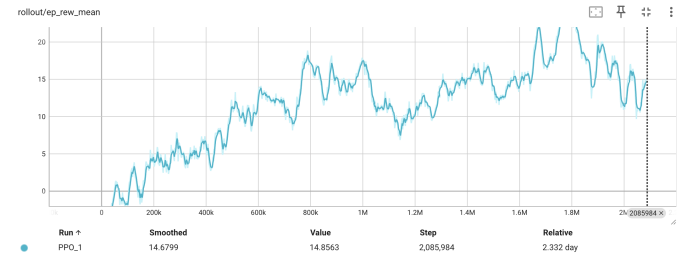


Fig. 6. Training curve. The horizontal axis is timesteps, and the vertical axis is mean reward. The curve near the end is allowed to oscillate in reinforcement learning and does not necessarily mean the model is performing worse.

The mean reward starts to go downward near the end of training. This is normal oscillation in reinforcement learning and does not necessarily mean the model is performing worse. However having an 83% win rate against a random-strategy opponent does not establish that the agent is competitive against skilled opponents. Further training against stronger opponents is required to achieve a deeper understanding of game mechanics. In future work, we plan to incorporate self-play training, which is expected to yield substantial performance gains.

VI. CONCLUSION

A key obstacle in Clash Royale AI research has been the absence of a publicly available simulator that can both run fast and support realistic game mechanics. We address this gap by introducing a faithful and fast Clash Royale Simulator, achieving a 49x speedup in experience collection while near-accurately reproducing core game mechanics, with 47 of 121 cards implemented and a complete, flexible framework for card behaviour. An RL environment is implemented based on the battle engine, with preliminary RL training demonstrating the environment’s viability, achieving an 83% win rate against a random opponent, confirming that the simulator constitutes a functional training environment.

VII. FUTURE WORK

There are three directions in future work ideas. In the near term, we plan to extend training to 10M timesteps and in-

corporate self-play, with the goal of approaching human-level performance. In the middle term, we also want to integrate more Clash Royale cards into the simulator so that the AI can be trained to play popular decks and evaluate their win rates. In the long term, we should explore how to read the game state using an android emulator directly from network traffic or memory, to make the agent actually train on real, authentic Clash Royale gameplay data and compete against human players.

REFERENCES

- [1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [2] C. Berner et al., “Dota 2 with large scale deep reinforcement learning,” *arXiv preprint arXiv:1912.06680*, 2019.
- [3] O. Vinyals et al., “Grandmaster level in StarCraft II using multi-agent reinforcement learning,” *Nature*, vol. 575, pp. 350–354, 2019.
- [4] ActivePlayer.io, “Clash Royale Live Player Count,” <https://activeplayer.io/clash-royale/>, accessed 2025.
- [5] R. Boney, A. Ilin, J. Kannala, and J. Seppänen, “Learning to Play Imperfect-Information Games by Imitating an Oracle Planner,” *arXiv preprint arXiv:2012.12186*, 2020.
- [6] Y. Liu et al., “Playing Card-Based RTS Games with Deep Reinforcement Learning,” in *Proc. IJCAI*, 2019.
- [7] Wu et al., “Playing Non-Embedded Card-Based Games with Reinforcement Learning,” *arXiv preprint arXiv:2504.04783*, 2025.